SimmerLink — Word-for-Word English Speaker Script & Jury Q&A Guide

Universitat Jaume I (UJI) · EI-1057 (*Mobile Networks & Devices*) · Companion Script for `UJI_EI1057_ENGLISH_PRESENTATION.pdf` (Slides 1–8)

Slide 1 — Title & Steve Jobs "Connecting the Dots" Hook

0:00 – 1:15

EYE CONTACT · SMILE "Good morning everyone. In his famous Stanford commencement address, Steve Jobs talked about **connecting the dots**—how seemingly separate experiences and technologies only make sense when you connect them into something meaningful."

"Picture a very common everyday problem: you are in the basement of a supermarket with **zero cellular coverage**, trying to coordinate a grocery list with your flatmate. Later, you are in the kitchen with flour on your hands: your mobile phone locks its screen, your background timer freezes, and you have no idea if your steak is at 56 degrees Celsius inside."

SIGNPOSTING ROADMAP "Today, I am presenting **SimmerLink**—an Android and iOS application that connects four core networking dots from **EI-1057**: **first**, our Offline-First SQLite core and Native OS Radio Diagnostics; **second**, our MANET UDP Broadcast discovery and Full-Duplex WebSocket P2P engine; **third**, our BLE 5.0 GATT binary telemetry protocol; and **finally**, our verification suite and visual walkthrough of the Smart Kitchen Wireless Hub."

Slide 2 — Mobile Context & Offline-First Core (Theory 1–2 & Lab 1)

1:15 – 2:30

SIGNPOSTING "Let us begin with the fundamental rule of mobile computing from **Theory 1 and Lab 1**: *mobile connectivity is intermittent, and the operating system aggressively suspends background processes.*"

POINT TO CARDS 1–3 "First, if a user walks into a supermarket basement without 5G or Wi-Fi, cloud-only apps fail. SimmerLink uses **Drift over SQLite** (`app_database.dart`), so every recipe, aisle-sorted ingredient, and portion scaling calculation runs locally at **zero bits per second**. Second, when an Android or iOS screen locks during cooking, naive periodic counters freeze. We solved this using **Wall-Clock Lifecycle Synchronization**: we persist the absolute `targetEndTime` timestamp and observe `AppLifecycleState.resumed` via `WidgetsBindingObserver`, recalculating exact remaining seconds the instant the phone wakes while keeping the display active in Cook Mode via `WakelockPlus`."

Slide 3 — Five Networking Layers Mapped to EI-1057 Syllabus

2:30 – 3:45

SIGNPOSTING "Moving on to our global network architecture, this table maps every wireless layer of **SimmerLink** directly to the **EI-1057 syllabus**."

"Instead of forcing one protocol to do everything, we matched each physical range and payload size to the optimal transport: at the **OS hardware layer**, our Kotlin and Swift `MethodChannel` inspects real radio link metrics; at the **WPAN layer** under 10 meters, **BLE 5.0 GATT** streams 2-byte and 3-byte binary sensor frames; at the **MANET layer**, connectionless **UDP Broadcast on port 41234** discovers nearby phones without manual IP entry; at the **WLAN layer**, **Full-Duplex WebSockets on port 8765** synchronize state in under 10 milliseconds; and at the **WWAN layer**, **HTTPS** connects to **Google Gemini 2.5 Flash** for multimodal fridge-photo computer vision."

Slide 4 — Native Android & iOS Network APIs (Theory 5 & Lab 5)

3:45 – 5:00

SIGNPOSTING "Let us now examine **Lab 5: Native Mobile OS Network APIs** in detail."

"Standard Dart sockets cannot read physical Wi-Fi link speed, carrier frequency bands, or Bluetooth adapter state, nor can they acquire an Android hardware `MulticastLock`. Therefore, in `MainActivity.kt` and `AppDelegate.swift`, I implemented a native bridge over `MethodChannel('com.mobkitchen/native_network')`."

"On Android, it queries `ConnectivityManager` and `NetworkCapabilities` for active transport and signal strength, `WifiManager` for link speed in Mbps, 2.4 vs. 5 GHz band classification, and `WifiManager.MulticastLock` (`simmerlink_udp_manet_lock`) so the Wi-Fi chipset does not drop incoming UDP broadcast packets, plus `BluetoothManager` for BLE radio state. In `AndroidManifest.xml`, we declare Android 12+ `NEARBY_WIFI_DEVICES` and `BLUETOOTH_SCAN` with `neverForLocation` to protect user GPS privacy. On iOS and our Mac Simulator, `AppDelegate.swift` inspects BSD POSIX `getifaddrs` on interface `en0`."

Slide 5 — MANET UDP Discovery (:41234) vs. Full-Duplex WebSocket (:8765)

5:00 – 6:30

SIGNPOSTING "This brings me to a core engineering question in **EI-1057 (Labs 3 and 4)**: *Why do we combine UDP and TCP/WebSockets in the same local network engine?*"

"In `lan_sync_engine.dart`, Phase 1 uses **UDP Broadcast on Port 41234** via `RawDatagramSocket`. TCP cannot broadcast to a subnet because it requires a unicast 3-way handshake; scanning 254 IPs via TCP takes seconds and triggers OS rate limits. Instead, every 4 seconds, each phone sends a single UDP datagram to `255.255.255.255:41234` and the directed `/24` broadcast address."

HIGHLIGHT LOOP PREVENTION "Once discovered, Phase 2 automatically upgrades to a persistent **Full-Duplex TCP WebSocket on Port 8765**. Crucially, if two phones discover each other simultaneously and open mutual WebSockets, a naive relay creates an infinite broadcast storm. We solved this by stamping every frame with a unique `_pkt_id` checked against a 256-entry `_seenPacketIds` ring buffer, alongside `_outboundSocketsByEndpoint` socket reuse."

Slide 6 — BLE 5.0 GATT Binary Framing & Physics Engine (Theory 6 & Lab 2) 6:30 – 7:45

SIGNPOSTING "Turning to the **WPAN IoT layer**: BLE 5.0 has a default ATT payload limit of 20 bytes, so sending verbose JSON wastes coin-cell battery energy. We implemented official **Bluetooth SIG GATT binary framing** using Dart `ByteData` and C++ ESP32 firmware (`esp32_smart_kitchen_node.ino`)."

"For our **Smart Meat Thermometer** (Service `0x181A`, Characteristic `0x2A6E`), temperature is encoded as a **2-byte signed 16-bit little-endian integer** at 0.01°C resolution—for example, `56.00°C` is `5600`, or hex `[0xE0, 0x15]`. For our **Smart Kitchen Scale** (Service `0x181D`, Characteristic `0x2A9D`), weight is encoded in **3 bytes**: a 1-byte SI flag (`0x00`) plus a 2-byte unsigned little-endian integer at 0.1-gram precision. We also built a deterministic physics engine modeling **Newton's Law of Heating** so every feature can be tested in the classroom."

Slide 7 — 3-Tab Visual Walkthrough & Verification Record 7:45 – 9:15

SIGNPOSTING · VISUAL WALKTHROUGH "Let us now review our verification results and walk through the three tabs of the **Smart Kitchen Wireless Hub** shown on Slide 7 (and running on our Simulator):"

1. Left Screen — Device Radar Tab (*Cihaz Radarı*): "At the bottom, the Native OS Network Diagnostics card invokes `com.mobkitchen/native_network` to display active `WIFI` transport, `-52 dBm` RSSI, `866 Mbps (5 GHz)` link speed, local IPv4 `192.168.1.42`, and active BLE connections."

2. Center Screen — Live Telemetry Tab (*Canlı Telemetri*): "Here we stream GATT `0x2A6E` temperature frames toward the **56°C Medium-Rare Steak** target with a live Newton heating curve and ETA countdown, alongside the GATT `0x2A9D` 0.1g Smart Scale with Tare calibration at `400.0 g`."

3. Right Screen — Wi-Fi Co-Op Tab: "Starting the Co-Op room binds both TCP WebSocket Port `8765` and MANET UDP Broadcast Port `41234`. The live Packet Log at the bottom records automatic UDP peer discovery, real-time Cook Mode step synchronization (`Step 2/4`), and live SQLite supermarket checkoffs. Across the entire repository, `flutter build apk --debug` succeeds cleanly, `flutter analyze` reports **0 issues**, and **120 out of 120 automated tests pass**."

Slide 8 — Conclusion, Rubric Summary & Opening the Floor to Q&A 9:15 – 10:00

SIGNPOSTING · CONCLUSION "To conclude and connect all the dots: **SimmerLink** demonstrates how offline-first SQLite persistence, native Android and iOS radio diagnostics, BLE 5.0 binary telemetry, MANET UDP zero-config discovery, and Full-Duplex WebSockets work together in a cohesive mobile system. As Steve Jobs famously concluded at Stanford—and as any chef would agree: **'Stay Hungry, Stay Foolish.'** Thank you very much for your attention. I am happy to answer any questions from the jury."

Jury Q&A Defense Bank (Prepared English Technical Answers)

Q1: "Why did you use UDP Broadcast on Port 41234 for discovery instead of scanning all 254 IP addresses with TCP?"

"Scanning a `/24` subnet with TCP requires up to 254 individual 3-way handshakes (`SYN`, `SYN-ACK`, `ACK`), which takes several seconds, drains mobile battery, and triggers OS firewall rate limits. By contrast, a single **UDP Broadcast datagram** to `255.255.255.255:41234` and the directed `/24` broadcast address (`192.168.x.255:41234`) via `RawDatagramSocket` (`broadcastEnabled = true`) reaches every peer on the local Wi-Fi AP in under 2 milliseconds. Once a peer parses the `simmer_beacon` JSON, it opens a single targeted TCP WebSocket connection to `ws://<peer-ip>:8765/ws`."

Q2: "What happens if two phones discover each other via UDP at the same time and both call connectToPeerWebSocket?"

"That creates two parallel WebSocket channels between Phone A and Phone B. Without protection, when Phone A relays an incoming packet to connected peers, Phone B would relay it back infinitely. In `LanSyncAndBleCodecEngine`, we prevented this in two ways: (1) every JSON frame is stamped with a unique `_pkt_id` checked against a bounded 256-entry `_seenPacketIds` ring buffer before processing or relaying, and (2) outbound client sockets are indexed in `_outboundSocketsByEndpoint['ip:port']` so periodic 4-second UDP beacons reuse the existing open WebSocket."

Q3: "How does your Android native MethodChannel handle Android 10+ and Android 12+ privacy and Wi-Fi hardware restrictions?"

"On Android 10 (API 29)+, `NetworkCapabilities.signalStrength` provides transport-layer RSSI directly from `ConnectivityManager`, supplemented by `WifiManager.connectionInfo` for link speed (Mbps) and 2.4/5 GHz frequency (MHz). Because Android Wi-Fi chipsets filter incoming UDP broadcast frames by default to save power, `MainActivity.kt` acquires a `WifiManager.MulticastLock` (`simmerlink_udp_manet_lock`). Finally, in `AndroidManifest.xml`, we declare `NEARBY_WIFI_DEVICES` and `BLUETOOTH_SCAN` with `android:usesPermissionFlags='neverForLocation'` so we never harvest user GPS location."

Q4: "Why encode BLE GATT telemetry in 2-byte and 3-byte Little-Endian binary instead of sending JSON over BLE?"

"The default BLE ATT MTU is 23 bytes (20 bytes of usable notification payload). A JSON string like `{\"temp_c\":56.00,\"weight_g\":125.4}` exceeds 32 bytes, requiring fragmentation and higher radio duty cycle on coin-cell probes. By following Bluetooth SIG GATT specifications—**`0x2A6E` (signed `int16` little-endian, 0.01°C)** and **`0x2A9D` (1-byte flags + `uint16` little-endian, 0.1g)**—our payloads are just **2 bytes and 3 bytes**, cutting radio airtime by over 90%."